

Greedy Algorithms: Part 1

Dr. Bugra Caskurlu¹

¹Computer Engineering Department
TOBB ETU University

February, 2015

Meaning of Greedy

Definition

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.

Meaning of Greedy

Definition

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.

Meaning of Greedy

Definition

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- When a greedy algorithm succeeds in solving a nontrivial problem optimally, it typically implies something interesting and useful about the structure of the problem itself; there is a local decision rule that one can use to construct optimal solutions.

Meaning of Greedy

Definition

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- When a greedy algorithm succeeds in solving a nontrivial problem optimally, it typically implies something interesting and useful about the structure of the problem itself; there is a local decision rule that one can use to construct optimal solutions.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**

Meaning of Greedy

Definition

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- When a greedy algorithm succeeds in solving a nontrivial problem optimally, it typically implies something interesting and useful about the structure of the problem itself; there is a local decision rule that one can use to construct optimal solutions.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**
- We will develop two basic methods for proving that a greedy algorithm produces an optimal solution to a problem. (One of them today, the other one in the next lecture).

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.
- To illustrate, if one measures the greedy algorithm's progress in a step-by-step fashion, one sees that *it does better than any algorithm at each step*.

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.
- To illustrate, if one measures the greedy algorithm's progress in a step-by-step fashion, one sees that *it does better than any algorithm at each step*.
- It then follows that our algorithm produces an optimal solution.

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.
- To illustrate, if one measures the greedy algorithm's progress in a step-by-step fashion, one sees that *it does better than any algorithm at each step*.
- It then follows that our algorithm produces an optimal solution.

Exchange Argument

- The second approach is known as the **exchange argument**, and it is more general.

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.
- To illustrate, if one measures the greedy algorithm's progress in a step-by-step fashion, one sees that *it does better than any algorithm at each step*.
- It then follows that our algorithm produces an optimal solution.

Exchange Argument

- The second approach is known as the **exchange argument**, and it is more general.
- One considers any possible solution to the problem and *gradually transforms it into the solution found by the greedy algorithm without hurting its quality*.

Proof Techniques

Greedy Stay Ahead

- Our first approach can be viewed as establishing that **the greedy algorithm stays ahead**.
- To illustrate, if one measures the greedy algorithm's progress in a step-by-step fashion, one sees that *it does better than any algorithm at each step*.
- It then follows that our algorithm produces an optimal solution.

Exchange Argument

- The second approach is known as the **exchange argument**, and it is more general.
- One considers any possible solution to the problem and *gradually transforms it into the solution found by the greedy algorithm without hurting its quality*.
- Then, it follows that the solution found by the greedy algorithm is at least as good as any other solution.

Interval Scheduling Problem

Informal Problem Definition

- You have a resource (a lecture room, a supercomputer, an electron microscope), and many people request to use the resource for periods of time.

Interval Scheduling Problem

Informal Problem Definition

- You have a resource (a lecture room, a supercomputer, an electron microscope), and many people request to use the resource for periods of time.
- A request takes the form: Can I reserve the resource starting at time s , until time f ?

Interval Scheduling Problem

Informal Problem Definition

- You have a resource (a lecture room, a supercomputer, an electron microscope), and many people request to use the resource for periods of time.
- A request takes the form: Can I reserve the resource starting at time s , until time f ?
- We will assume that the resource can be used by at most one person at a time.

Interval Scheduling Problem

Informal Problem Definition

- You have a resource (a lecture room, a supercomputer, an electron microscope), and many people request to use the resource for periods of time.
- A request takes the form: Can I reserve the resource starting at time s , until time f ?
- We will assume that the resource can be used by at most one person at a time.
- A scheduler wants to accept a subset of these requests, rejecting all others, so that the accepted requests do not overlap in time.

Interval Scheduling Problem

Informal Problem Definition

- You have a resource (a lecture room, a supercomputer, an electron microscope), and many people request to use the resource for periods of time.
- A request takes the form: Can I reserve the resource starting at time s , until time f ?
- We will assume that the resource can be used by at most one person at a time.
- A scheduler wants to accept a subset of these requests, rejecting all others, so that the accepted requests do not overlap in time.
- The goal is to maximize the number of requests accepted.

Interval Scheduling

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .

Interval Scheduling

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$.

Interval Scheduling

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$.
- We'll say more generally that a subset A of requests are compatible if all pairs of requests $i, j \in A$, $i \neq j$ are compatible.

Interval Scheduling

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$.
- We'll say more generally that a subset A of requests are compatible if all pairs of requests $i, j \in A$, $i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size.

Interval Scheduling

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$.
- We'll say more generally that a subset A of requests are compatible if all pairs of requests $i, j \in A$, $i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size.
- Compatible sets of maximum size will be called *optimum*.

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 .

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 . Once a request i_1 is accepted, we reject all requests that are not compatible with i_1 .

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 . Once a request i_1 is accepted, we reject all requests that are not compatible with i_1 .
- We then select the next request i_2 to be accepted, and again reject all requests that are not compatible with i_2 .

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 . Once a request i_1 is accepted, we reject all requests that are not compatible with i_1 .
- We then select the next request i_2 to be accepted, and again reject all requests that are not compatible with i_2 .
- We continue in this fashion until we run out of requests.

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 . Once a request i_1 is accepted, we reject all requests that are not compatible with i_1 .
- We then select the next request i_2 to be accepted, and again reject all requests that are not compatible with i_2 .
- We continue in this fashion until we run out of requests.
- **The challenge in designing a good greedy algorithm is in deciding which simple rule to use for the selection.**

Designing a Greedy Algorithm

General Technique

- The basic idea in a greedy algorithm for interval scheduling is to use a simple rule to select a first request i_1 . Once a request i_1 is accepted, we reject all requests that are not compatible with i_1 .
- We then select the next request i_2 to be accepted, and again reject all requests that are not compatible with i_2 .
- We continue in this fashion until we run out of requests.
- **The challenge in designing a good greedy algorithm is in deciding which simple rule to use for the selection.**
- *There are typically many natural rules that do not give good solutions.*

Some Bad Rules

Earliest Starting Request Rule

- **Select the available request that starts earliest, i.e., the one with the minimal start time $s(i)$.** This way our resource starts being used a.s.a.p.

Some Bad Rules

Earliest Starting Request Rule

- **Select the available request that starts earliest, i.e., the one with the minimal start time $s(i)$.** This way our resource starts being used a.s.a.p.
- *Not returns optimal solution.* if the earliest request i is for a very long interval, by accepting request i we may have to reject a lot of requests.

Some Bad Rules

Earliest Starting Request Rule

- **Select the available request that starts earliest, i.e., the one with the minimal start time $s(i)$.** This way our resource starts being used a.s.a.p.
- *Not returns optimal solution.* if the earliest request i is for a very long interval, by accepting request i we may have to reject a lot of requests.

Shortest Request Rule

- At each step, select the available request that requires the smallest amount time, i.e., **the one for which $f(i) - s(i)$ is smallest.**

Some Bad Rules

Earliest Starting Request Rule

- **Select the available request that starts earliest, i.e., the one with the minimal start time $s(i)$.** This way our resource starts being used a.s.a.p.
- *Not returns optimal solution.* if the earliest request i is for a very long interval, by accepting request i we may have to reject a lot of requests.

Shortest Request Rule

- At each step, select the available request that requires the smallest amount time, i.e., **the one for which $f(i) - s(i)$ is smallest.**
- *Not returns optimal solution.* Consider the set of requests $\{(1, 5), (6, 10), (4, 7)\}$.

Some Bad Rules

Earliest Starting Request Rule

- **Select the available request that starts earliest, i.e., the one with the minimal start time $s(i)$.** This way our resource starts being used a.s.a.p.
- *Not returns optimal solution.* if the earliest request i is for a very long interval, by accepting request i we may have to reject a lot of requests.

Shortest Request Rule

- At each step, select the available request that requires the smallest amount time, i.e., **the one for which $f(i) - s(i)$ is smallest.**
- *Not returns optimal solution.* Consider the set of requests $\{(1, 5), (6, 10), (4, 7)\}$.
- In this example, accepting the short interval in the middle prevented us from accepting the other two, which form an optimal solution. The problem in this example is that the short interval is incompatible with all other intervals.

Some More Rules

Fewest Conflicts Rule

- For each request, count the number of other requests that are not compatible, and accept the request that has the fewest number of noncompatible requests.

Some More Rules

Fewest Conflicts Rule

- For each request, count the number of other requests that are not compatible, and accept the request that has the fewest number of noncompatible requests.
- *Not returns optimal solution. Can you give an example?*

Some More Rules

Fewest Conflicts Rule

- For each request, count the number of other requests that are not compatible, and accept the request that has the fewest number of noncompatible requests.
- *Not returns optimal solution. Can you give an example?*

Good Rule

- **Accept the requests that finishes first**, i.e., accept the request for which $f(i)$ is as small as possible.

Some More Rules

Fewest Conflicts Rule

- For each request, count the number of other requests that are not compatible, and accept the request that has the fewest number of noncompatible requests.
- *Not returns optimal solution. Can you give an example?*

Good Rule

- **Accept the requests that finishes first**, i.e., accept the request for which $f(i)$ is as small as possible.
- We ensure that our resource becomes free a.s.a.p. while still satisfying one request.

Some More Rules

Fewest Conflicts Rule

- For each request, count the number of other requests that are not compatible, and accept the request that has the fewest number of noncompatible requests.
- *Not returns optimal solution. Can you give an example?*

Good Rule

- **Accept the requests that finishes first**, i.e., accept the request for which $f(i)$ is as small as possible.
- We ensure that our resource becomes free a.s.a.p. while still satisfying one request. In this way, we can maximize the time left to satisfy the other requests.

Earliest Finishing Rule

Pseudocode

Initially let R be the set of all requests, and let A be empty

while R is not empty

 Choose request $i \in R$ with the smallest finishing time

 Add request i to A

 Delete all requests noncompatible with i from R

return the set A as the set of accepted requests

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.
- **Feasibility:** A is a compatible set of request.

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.
- **Feasibility:** A is a compatible set of request.
- **Optimality:** For the purpose of comparison, let O be an optimal set of intervals. Optimality condition is that $|A| = |O|$.

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.
- **Feasibility:** A is a compatible set of request.
- **Optimality:** For the purpose of comparison, let O be an optimal set of intervals. Optimality condition is that $|A| = |O|$.

Defining Notation

- Let $i_1, i_2, \dots, i_k$ be the set of requests in $\mathbf{A}$ in the order they were added to $\mathbf{A}$. Note that $|A| = k$.

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.
- **Feasibility:** A is a compatible set of request.
- **Optimality:** For the purpose of comparison, let O be an optimal set of intervals. Optimality condition is that $|A| = |O|$.

Defining Notation

- Let $i_1, i_2, \dots, i_k$ be the set of requests in A in the order they were added to A . Note that $|A| = k$.
- Let the set of requests in O be denoted by $j_1, j_2, \dots, j_m$. Assume that the requests in O are also ordered in the natural left-to-right order of the corresponding intervals, i.e., in the order of the start and finish points. Since O is compatible start points have the same order as the finish points.

Proving the Algorithm

What needs to be Proven?

- When you give an algorithm for an optimization problem, you need to prove two things.
- **Feasibility:** A is a compatible set of request.
- **Optimality:** For the purpose of comparison, let O be an optimal set of intervals. Optimality condition is that $|A| = |O|$.

Defining Notation

- Let $i_1, i_2, \dots, i_k$ be the set of requests in A in the order they were added to A . Note that $|A| = k$.
- Let the set of requests in O be denoted by $j_1, j_2, \dots, j_m$. Assume that the requests in O are also ordered in the natural left-to-right order of the corresponding intervals, i.e., in the order of the start and finish points. Since O is compatible start points have the same order as the finish points.
- Our goal is to prove that $k = m$.

Stay Ahead Property

Intuition

- Our intuition for the greedy method came from wanting our resource to become free a.s.a.p. after satisfying the first request.

Stay Ahead Property

Intuition

- Our intuition for the greedy method came from wanting our resource to become free a.s.a.p. after satisfying the first request.
- Our greedy rule guarantees that $f(i_1) \leq f(j_1)$.

Stay Ahead Property

Intuition

- Our intuition for the greedy method came from wanting our resource to become free a.s.a.p. after satisfying the first request.
- Our greedy rule guarantees that $f(i_1) \leq f(j_1)$.
- This is the sense in which we want to show that our greedy rule *stays ahead*, i.e., **each of its intervals finishes at least as soon as the corresponding interval in the set O .**

Stay Ahead Property

Intuition

- Our intuition for the greedy method came from wanting our resource to become free a.s.a.p. after satisfying the first request.
- Our greedy rule guarantees that $f(i_1) \leq f(j_1)$.
- This is the sense in which we want to show that our greedy rule *stays ahead*, i.e., **each of its intervals finishes at least as soon as the corresponding interval in the set O .**
- So, we will prove by induction that for each $r \geq 1$, the r^{th} accepted request in the algorithm's schedule finishes no later than the r^{th} request in the optimal schedule.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .
- **Inductive Step:** By inductive hypothesis, we have $f(i_{r-1}) \leq f(j_{r-1})$.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .
- **Inductive Step:** By inductive hypothesis, we have $f(i_{r-1}) \leq f(j_{r-1})$. Since O consists of compatible intervals, we have $f(j_{r-1}) \leq s(j_r)$.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .
- **Inductive Step:** By inductive hypothesis, we have $f(i_{r-1}) \leq f(j_{r-1})$. Since O consists of compatible intervals, we have $f(j_{r-1}) \leq s(j_r)$. Combining these two inequalities, we get $f(i_{r-1}) \leq s(j_r)$.

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .
- **Inductive Step:** By inductive hypothesis, we have $f(i_{r-1}) \leq f(j_{r-1})$. Since O consists of compatible intervals, we have $f(j_{r-1}) \leq s(j_r)$. Combining these two inequalities, we get $f(i_{r-1}) \leq s(j_r)$.
- Thus, the interval j_r is in the set R of available intervals at the time when the greedy algorithm selects i_r .

Proving Stay Ahead Property

Lemma

For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$.

Proof

- **Base Case:** For $r = 1$, the statement is trivially true: algorithm starts by selecting the request i_1 with the minimum finish time.
- **Inductive Hypothesis:** Let $r > 1$. Assume our inductive hypothesis is true for $r - 1$, and prove it for r .
- **Inductive Step:** By inductive hypothesis, we have $f(i_{r-1}) \leq f(j_{r-1})$. Since O consists of compatible intervals, we have $f(j_{r-1}) \leq s(j_r)$. Combining these two inequalities, we get $f(i_{r-1}) \leq s(j_r)$.
- Thus, the interval j_r is in the set R of available intervals at the time when the greedy algorithm selects i_r . The greedy algorithm selects the available interval with the **smallest finish time**, and since interval j_r is one of these available intervals, we have $f(i_r) \leq f(j_r)$ as desired. □

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$.

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .
- Since O is compatible, we have $f(j_k) \leq s(j_{k+1})$.

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .
- Since O is compatible, we have $f(j_k) \leq s(j_{k+1})$. By the above lemma, we have $f(i_k) \leq f(j_k)$.

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .
- Since O is compatible, we have $f(j_k) \leq s(j_{k+1})$. By the above lemma, we have $f(i_k) \leq f(j_k)$. By combining inequalities, we get $f(i_k) \leq s(j_{k+1})$.

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .
- Since O is compatible, we have $f(j_k) \leq s(j_{k+1})$. By the above lemma, we have $f(i_k) \leq f(j_k)$. By combining inequalities, we get $f(i_k) \leq s(j_{k+1})$.
- So after deleting all request that are compatible with requests $i_1, \dots, i_k$, the set of possible requests R still contains j_{k+1} .

Proving Optimality

Theorem

The greedy algorithm returns an optimal set A .

Proof

- For the purpose of contradiction, assume A is not optimal, i.e., $m > k$. There is a request j_{k+1} in O .
- Since O is compatible, we have $f(j_k) \leq s(j_{k+1})$. By the above lemma, we have $f(i_k) \leq f(j_k)$. By combining inequalities, we get $f(i_k) \leq s(j_{k+1})$.
- So after deleting all request that are compatible with requests $i_1, \dots, i_k$, the set of possible requests R still contains j_{k+1} .
- But the greedy algorithm stops with request i_k , and it is only supposed to stop when R is empty! Contradiction. □

Implementation and Running Time

Analysis of Algorithm

- We begin by sorting the n requests in order of finishing time and labeling them in this order. That is, we will assume that $f(i) \leq f(j)$ when $i < j$. **This takes time $O(n \cdot \lg n)$.**

Implementation and Running Time

Analysis of Algorithm

- We begin by sorting the n requests in order of finishing time and labeling them in this order. That is, we will assume that $f(i) \leq f(j)$ when $i < j$. **This takes time** $O(n \cdot \lg n)$.
- We now select requests by processing the intervals in order of increasing $f(i)$. We always select the first interval; we then iterate through the intervals in order until reaching the first interval j for which $s(j) \geq f(1)$.

Implementation and Running Time

Analysis of Algorithm

- We begin by sorting the n requests in order of finishing time and labeling them in this order. That is, we will assume that $f(i) \leq f(j)$ when $i < j$. **This takes time** $O(n \cdot \lg n)$.
- We now select requests by processing the intervals in order of increasing $f(i)$. We always select the first interval; we then iterate through the intervals in order until reaching the first interval j for which $s(j) \geq f(1)$.
- More generally, if the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$.

Implementation and Running Time

Analysis of Algorithm

- We begin by sorting the n requests in order of finishing time and labeling them in this order. That is, we will assume that $f(i) \leq f(j)$ when $i < j$. **This takes time $O(n \cdot \lg n)$.**
- We now select requests by processing the intervals in order of increasing $f(i)$. We always select the first interval; we then iterate through the intervals in order until reaching the first interval j for which $s(j) \geq f(1)$.
- More generally, if the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$.
- In this way, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. Thus, **this part takes $O(n)$ time.**

Implementation and Running Time

Analysis of Algorithm

- We begin by sorting the n requests in order of finishing time and labeling them in this order. That is, we will assume that $f(i) \leq f(j)$ when $i < j$. **This takes time $O(n \cdot \lg n)$.**
- We now select requests by processing the intervals in order of increasing $f(i)$. We always select the first interval; we then iterate through the intervals in order until reaching the first interval j for which $s(j) \geq f(1)$.
- More generally, if the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$.
- In this way, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. Thus, **this part takes $O(n)$ time.**
- Thus, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Interval Partitioning Problem

Definition

- In the Interval Scheduling Problem, there is a single resource and many requests in the form of time intervals, so we must choose which requests to accept and which to reject.

Interval Partitioning Problem

Definition

- In the Interval Scheduling Problem, there is a single resource and many requests in the form of time intervals, so we must choose which requests to accept and which to reject.
- A related problem arises if we have many identical resources and we will to schedule *all* the requests using as few resources as possible.

Interval Partitioning Problem

Definition

- In the Interval Scheduling Problem, there is a single resource and many requests in the form of time intervals, so we must choose which requests to accept and which to reject.
- A related problem arises if we have many identical resources and we will to schedule *all* the requests using as few resources as possible.
- Because the goal here is to partition all the intervals across multiple resources, we will refer to this as the Interval Partitioning Problem (or Interval Coloring Problem).

Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.

Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.
- If you can give an algorithm that returns a solution whose value matches the lower bound for all instances of the problem, then your algorithm is optimal!



Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.
- If you can give an algorithm that returns a solution whose value matches the lower bound for all instances of the problem, then your algorithm is optimal!

Lower Bound for Interval Partitioning

- Let the *depth* of a set of intervals be the maximum number of requests at any single point in the time-line.



Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.
- If you can give an algorithm that returns a solution whose value matches the lower bound for all instances of the problem, then your algorithm is optimal!

Lower Bound for Interval Partitioning

- Let the *depth* of a set of intervals be the maximum number of requests at any single point in the time-line.
- **(Structural) Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth of the set of intervals.*

Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.
- If you can give an algorithm that returns a solution whose value matches the lower bound for all instances of the problem, then your algorithm is optimal!

Lower Bound for Interval Partitioning

- Let the *depth* of a set of intervals be the maximum number of requests at any single point in the time-line.
- **(Structural) Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth of the set of intervals.*
- **Proof:** Suppose a set of intervals has depth d , and let $I_1, \dots, I_d$ all pass over a common point on the time-line. Then each of these intervals must be scheduled on a different resource, so the whole instance needs at least d intervals. □

Obtaining a Lower Bound

Technique

- When solving an optimization problem, sometimes you obtain a lower bound on the value of the optimal solution.
- If you can give an algorithm that returns a solution whose value matches the lower bound for all instances of the problem, then your algorithm is optimal!

Lower Bound for Interval Partitioning

- Let the *depth* of a set of intervals be the maximum number of requests at any single point in the time-line.
- **(Structural) Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth of the set of intervals.*
- **Proof:** Suppose a set of intervals has depth d , and let $I_1, \dots, I_d$ all pass over a common point on the time-line. Then each of these intervals must be scheduled on a different resource, so the whole instance needs at least d intervals. □
- Thus, if we can design an algorithm that returns a schedule using a number of resources that is equal to the depth for all instances, then our algorithm is optimal!

Interval Partitioning Problem

Pseudocode

Sort the intervals by their start times, break ties arbitrarily

Let $I_1, \dots, I_n$ denote the intervals in this order

For $j = 1, 2, 3, \dots, n$

 For each interval I_i that precedes I_j and overlaps it

 Exclude the label of I_i from consideration for I_j

 If there is any nonexcluded label from $1, 2, \dots, d$

 Assign a nonexcluded label to I_j

 Else

 Leave I_j unlabeled

Analysis of Algorithm

Feasibility

- **Claim:** *No two overlapping intervals are assigned the same label.*

Analysis of Algorithm

Feasibility

- **Claim:** *No two overlapping intervals are assigned the same label.*
- Consider any two intervals I and I' that overlap, and suppose I precedes I' in sorted order. Then, when I' is considered by the algorithm, I is in the set of intervals whose labels are excluded from consideration. Thus, the algorithm will not assign to I' the label it used for I . □.

Analysis of Algorithm

Feasibility

- **Claim:** *No two overlapping intervals are assigned the same label.*
- Consider any two intervals I and I' that overlap, and suppose I precedes I' in sorted order. Then, when I' is considered by the algorithm, I is in the set of intervals whose labels are excluded from consideration. Thus, the algorithm will not assign to I' the label it used for I . □.

Optimality

- **Claim:** *Each interval will be assigned a label.*

Analysis of Algorithm

Feasibility

- **Claim:** *No two overlapping intervals are assigned the same label.*
- Consider any two intervals I and I' that overlap, and suppose I precedes I' in sorted order. Then, when I' is considered by the algorithm, I is in the set of intervals whose labels are excluded from consideration. Thus, the algorithm will not assign to I' the label it used for I . □.

Optimality

- **Claim:** *Each interval will be assigned a label.*
- Consider one of the intervals I_j , and suppose there are t intervals earlier in the sorted order that overlap it. These t intervals together with I_j , form a set of $(t + 1)$ intervals that all pass over a common point on the time-line

Analysis of Algorithm

Feasibility

- **Claim:** *No two overlapping intervals are assigned the same label.*
- Consider any two intervals I and I' that overlap, and suppose I precedes I' in sorted order. Then, when I' is considered by the algorithm, I is in the set of intervals whose labels are excluded from consideration. Thus, the algorithm will not assign to I' the label it used for I . □.

Optimality

- **Claim:** *Each interval will be assigned a label.*
- Consider one of the intervals I_j , and suppose there are t intervals earlier in the sorted order that overlap it. These t intervals together with I_j , form a set of $(t + 1)$ intervals that all pass over a common point on the time-line (namely, the start time of I_j). So, $t + 1 \leq d$. Thus, $t \leq d - 1$. Hence, at least one label is available for I_j . □

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.
- We will assume that the price growth rates are distinct, i.e., $r_i \neq r_j$ for licences $i \neq j$ (even though they start at the same price of 100 TL).

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.
- We will assume that the price growth rates are distinct, i.e., $r_i \neq r_j$ for licences $i \neq j$ (even though they start at the same price of 100 TL).
- **Question:** *In which order should the company buy the licences so that the total amount of money it spends is as small as possible?*